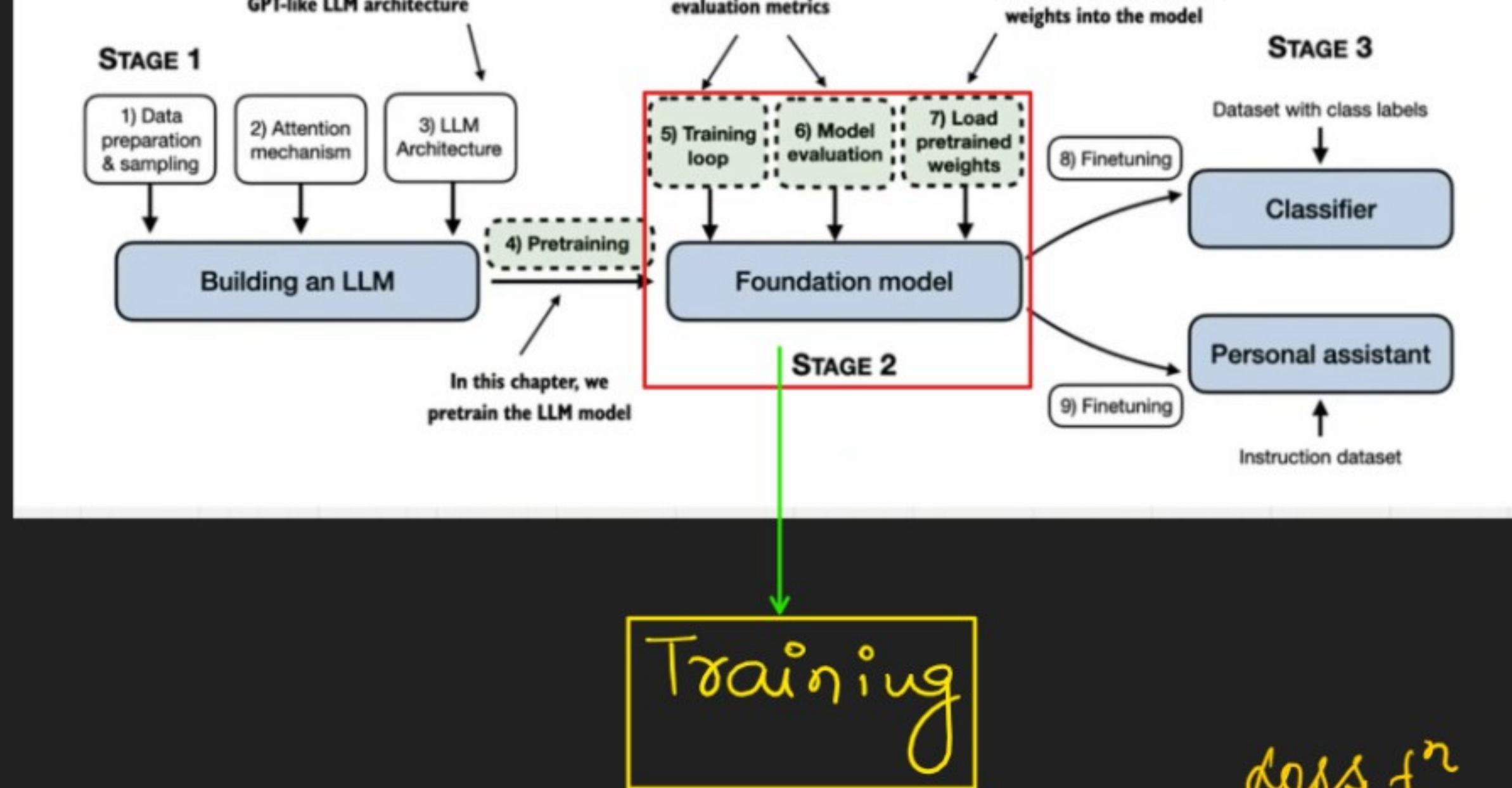


lecture-26: Pretraining LLM:

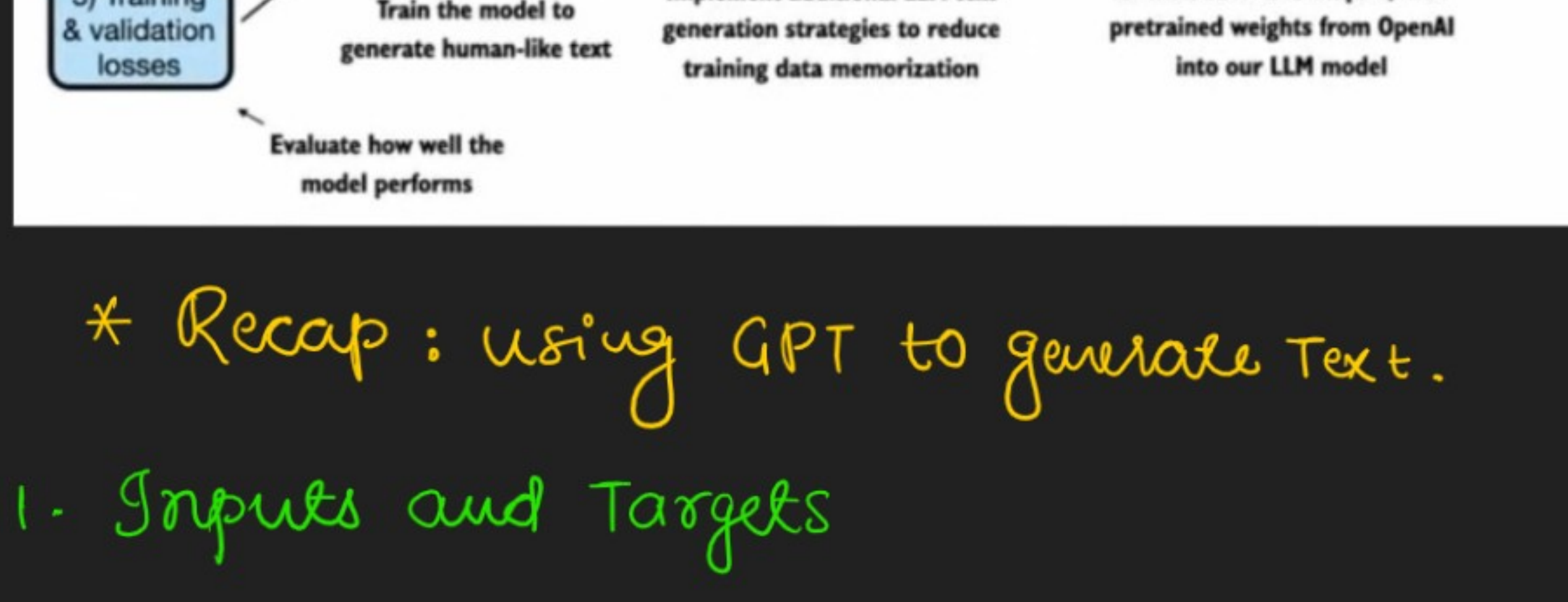
loss Function



Training

loss fn

1. How to measure LLM prediction loss?



* Recap: using GPT to generate Text.

1. Inputs and Targets

inputs = torch.tensor([[16833, 3626, 6100], # ['every effort moves']
[40, 1107, 588]]) # ['I really like']

matching these inputs, the `target` contain the token IDs we aim for the

targets = torch.tensor([[3626, 6100, 345], # ['effort moves you']
[107, 588, 11311]]) # ['really like chocolate']

these prediction.

2. Output

Input → GPT model → output token IDs

GPT MODEL

output tensor → logits

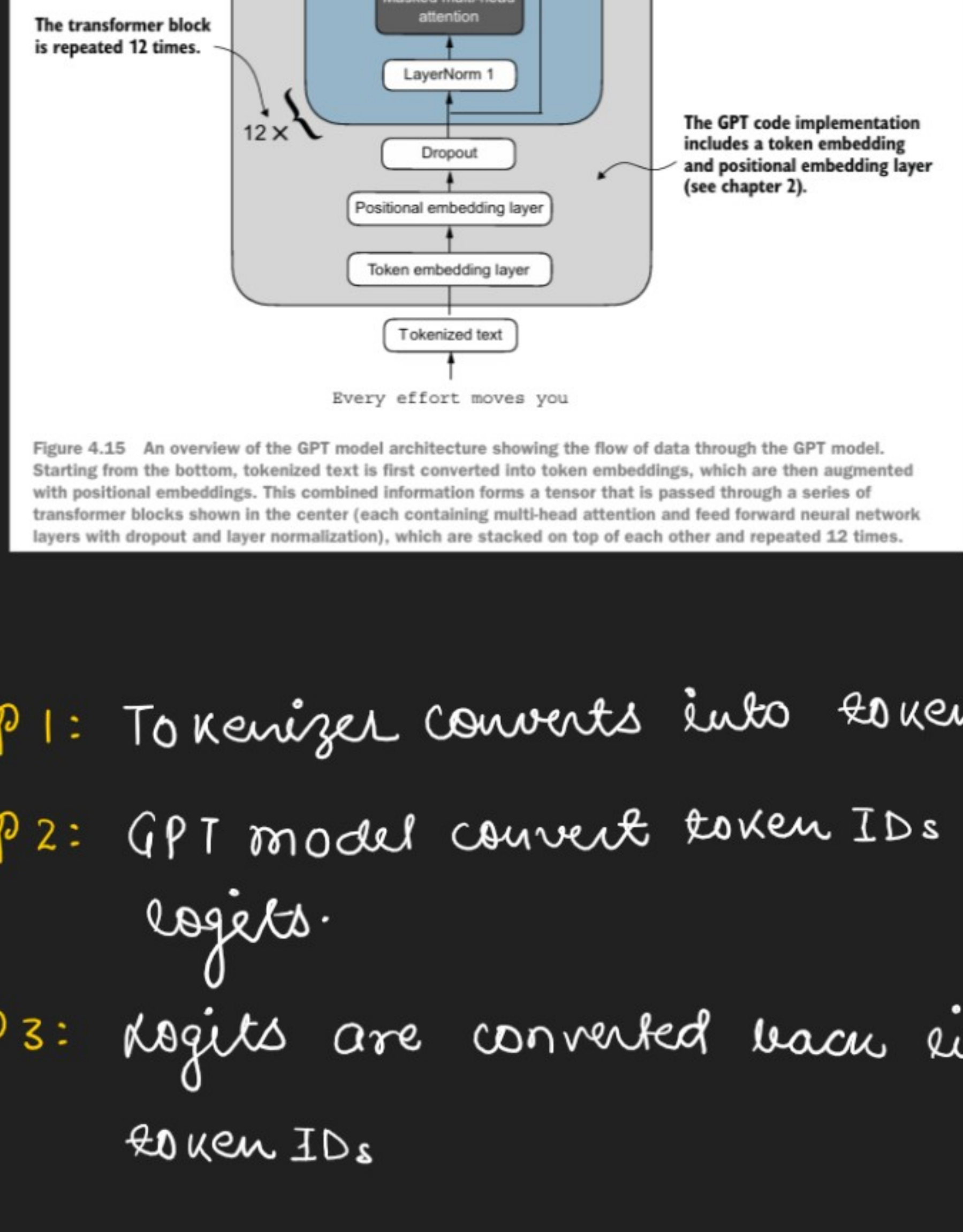


Figure 4.15 An overview of the GPT model architecture showing the flow of data through the GPT model.

Starting from the bottom, tokenized text is first converted into token embeddings, which are then augmented with positional embeddings. This combined information forms a tensor that is passed through a series of transformer blocks shown in the center (each containing multi-head attention and feed forward neural network layers with dropout and layer normalization), which are stacked on top of each other and repeated 12 times.

Step 1: Tokenizer converts into token IDs

Step 2: GPT model convert token IDs into logits.

Step 3: logits are converted back into token IDs

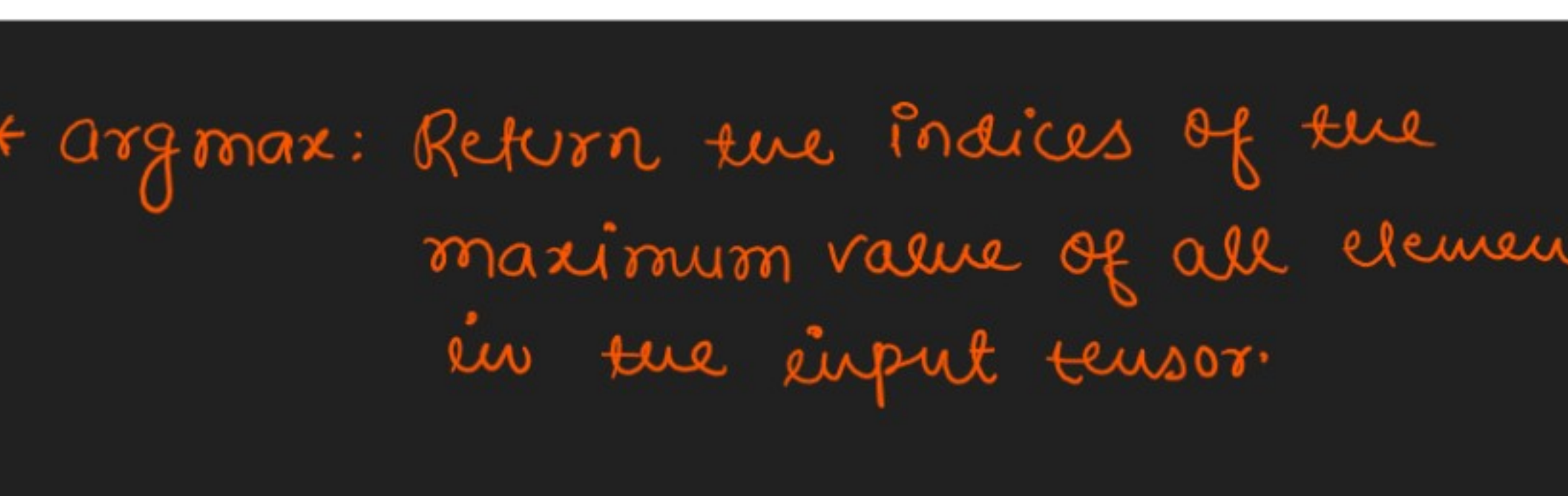


Figure 5.4 For each of the three input tokens, shown on the left, we compute a vector containing probability scores corresponding to each token in the vocabulary. The index position of the highest probability score in each vector represents the most likely next token ID. These token IDs associated with the highest probability scores are selected and mapped back into a text that represents the text generated by the model.

* Argmax: Return the indices of the maximum value of all elements in the input tensor.

Output token IDs

Token IDs:
every [16657],
effort [339],
moves [42826],
I [49906],
Really [29669],
like [41751]]

3. Finding the loss between targets & outputs

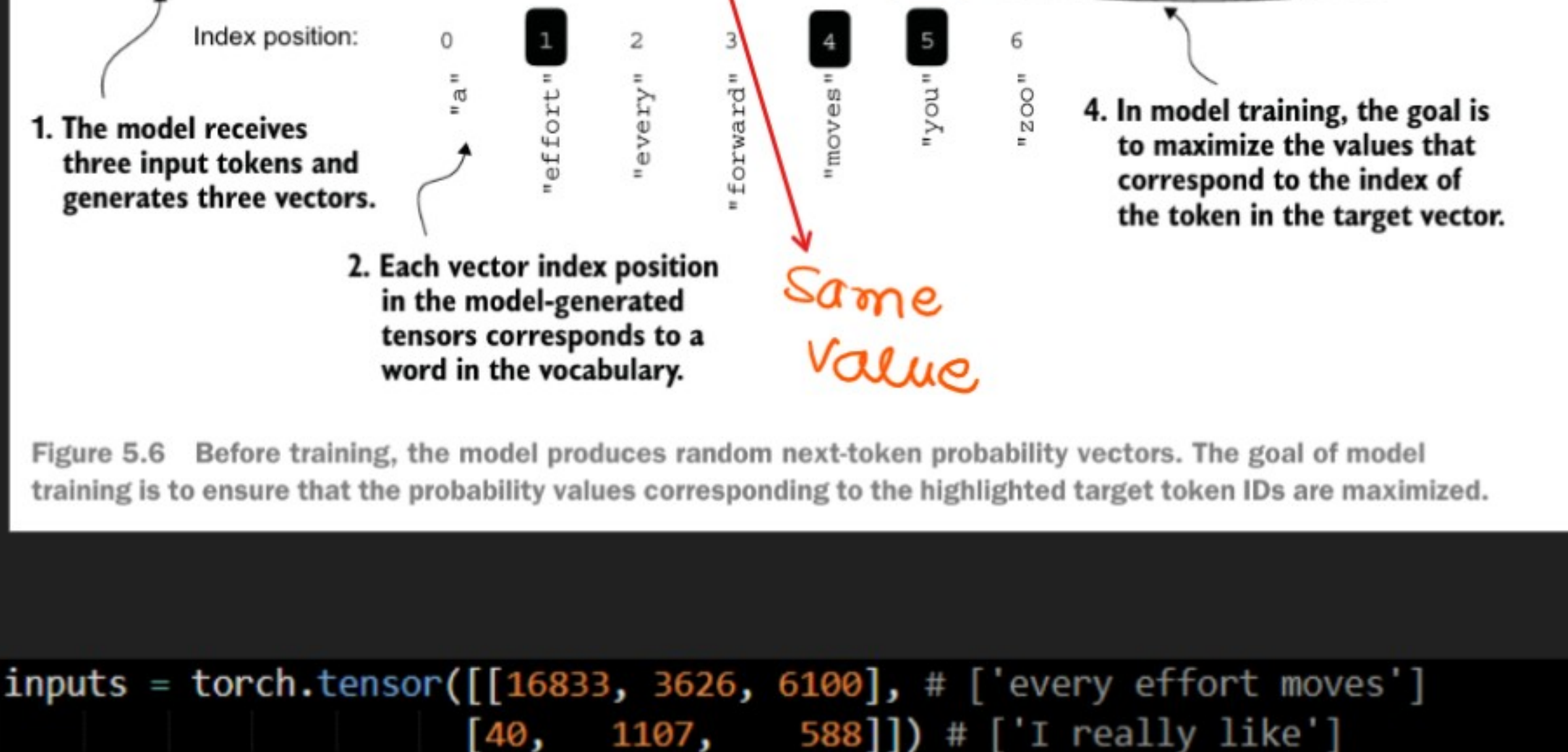


Figure 5.6 Before training, the model produces random next-token probability vectors. The goal of model training is to ensure that the probability values corresponding to the highlighted target token IDs are maximized.

inputs = torch.tensor([[16833, 3626, 6100], # ['every effort moves']
[40, 1107, 588]]) # ['I really like']

matching these inputs, the `target` contain the token IDs we aim for the

targets = torch.tensor([[3626, 6100, 345], # ['effort moves you']
[107, 588, 11311]]) # ['really like chocolate']

Target Indices:

Batch 1: [i₁₁, i₁₂, i₁₃]

Batch 2: [i₂₁, i₂₂, i₂₃]

targets = torch.tensor([[3626, 6100, 345], # ['effort moves you']
[107, 588, 11311]]) # ['really like chocolate']

Total probabilities:

Batch 1: [p₁₁, p₁₂, p₁₃]

Batch 2: [p₂₁, p₂₂, p₂₃]

Goal of training is to get these values as close to 1 as possible

Now merge them;

[p₁₁, p₁₂, p₁₃, p₂₁, p₂₂, p₂₃]

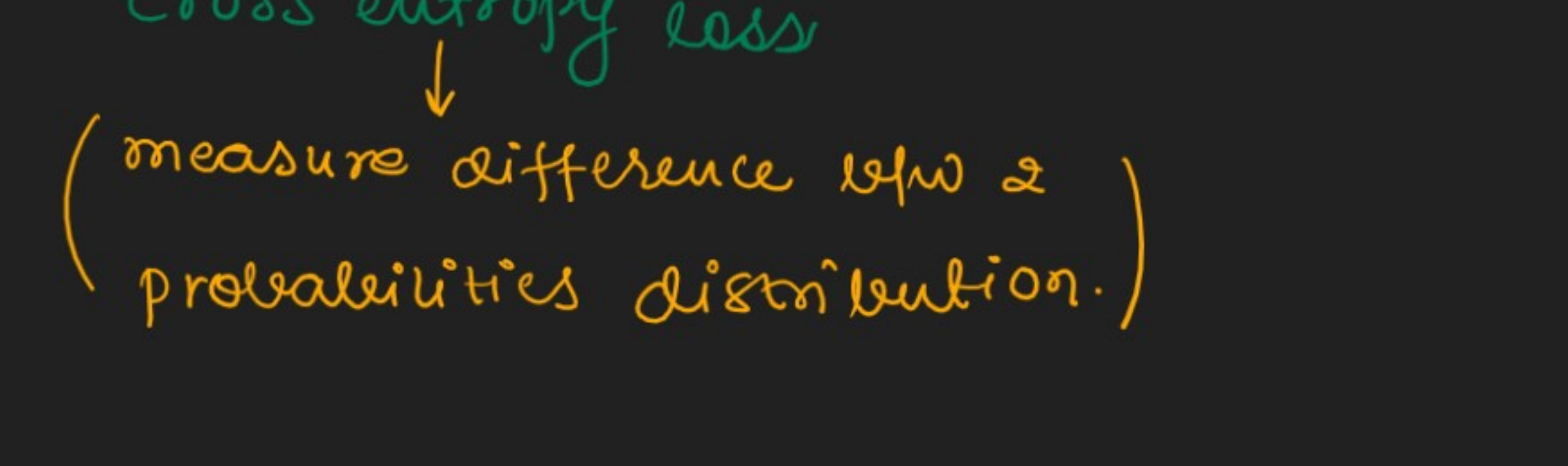


Figure 5.7 Calculating the loss involves several steps. Steps 1 to 3, which we have already completed, calculate the token probabilities corresponding to the target tensors. These probabilities are then transformed via a logarithm and averaged in steps 4 to 6.

Cross entropy loss

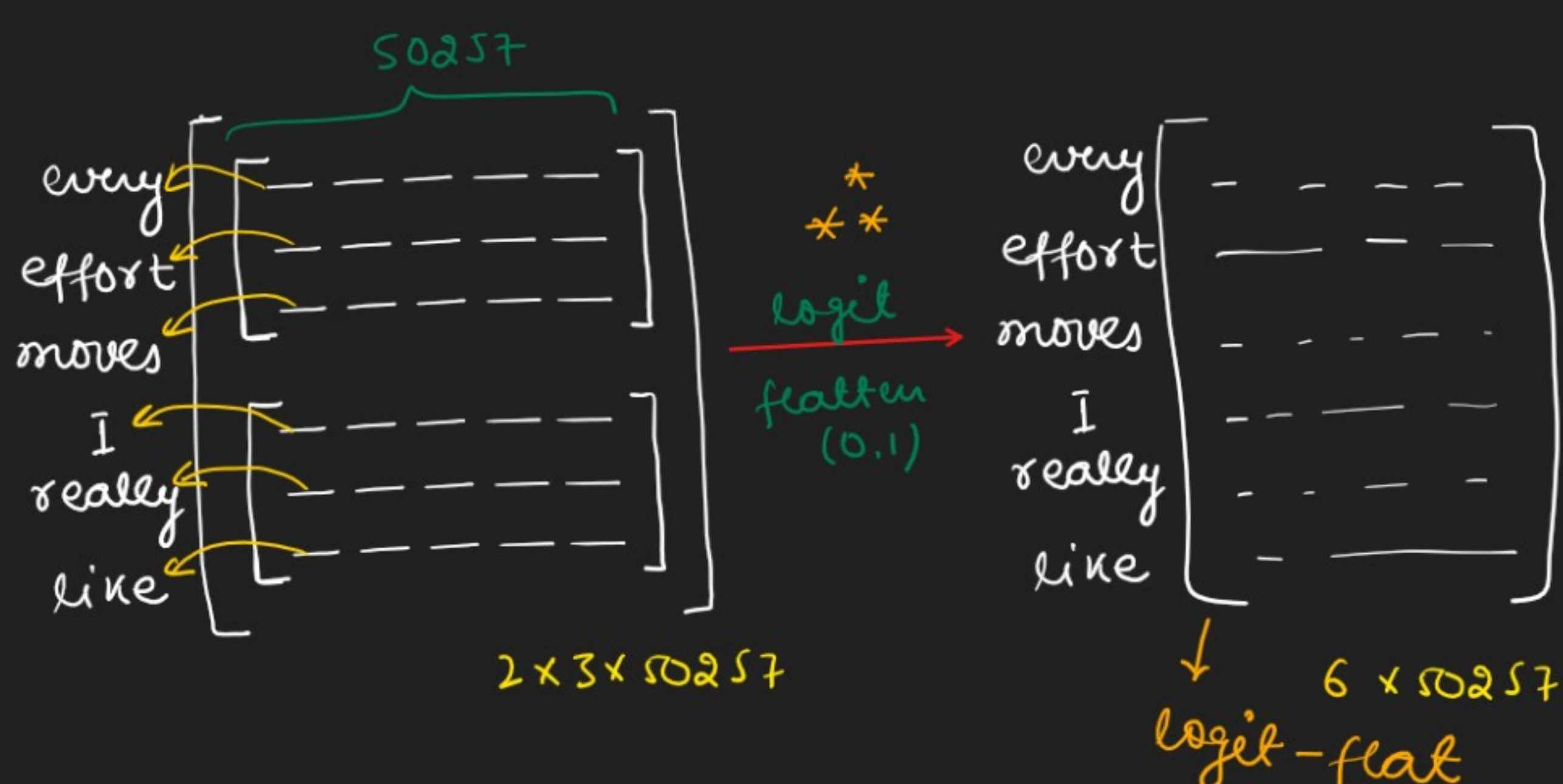
(measure difference b/w 2 probabilities distribution.)

loss = -log(x)

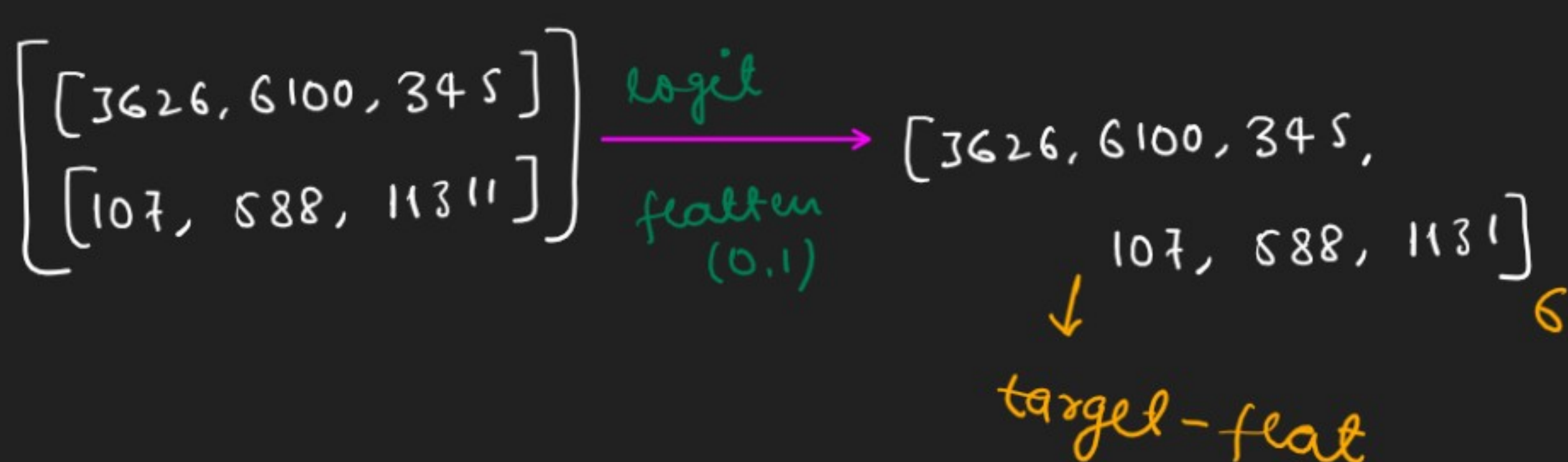
Bring down the loss to zero.

Cross entropy loss

* logits



* Targets



torch.nn.functional

• $\text{cross-entropy}[\text{logit-flat}, \text{target-flat}]$ } loss

- ① softmax to logit
 - ② Negative log likelihood
- ↳ b/w output and target

4. perplexity: Another loss measure like cross entropy.

→ Measure how well the probability distribution predicted by the model matches the actual distribution of words in the datasets.

→ More interpretable way of understanding model uncertainty in predicting next token.

→ lower perplexity score = Better prediction.

$$\text{perplexity} = \text{torch.exp}(\underbrace{\text{loss}}_{e^{\text{loss}}}) = 48725$$

model is roughly as uncertain as if it had to choose the next token randomly from about 48725 tokens in the vocabulary

much more interpretable